

Project Test Report: AnomalyDetection

This report reproduces the examples and experiments of the AnomalyDetection project by following the step-by-step instructions in the [README](#). The testing results were reproduced on an A100 GPU server.

Outline

- [Project Test Report: AnomalyDetection](#)
 - [Outline](#)
 - [1. Hardware and Software Specification](#)
 - [2. Evaluation Reproduction](#)
 - [2.1 Examples](#)
 - [2.2 Technical and Overhead Metrics](#)
 - [2.2.1 Technical Metrics: Image Level](#)
 - [2.2.2 Technical Metrics: Pixel Level](#)
 - [2.2.3 Technical Metrics: Instance Level](#)
 - [2.2.4 Overhead Metrics: Compute](#)
 - [2.2.5 Overhead Metrics: Memory](#)
 - [2.3 Analysis](#)
 - [3. Extensibility](#)
 - [3.1 Adding a New Dataset](#)
 - [3.2 Adding the YOLO 26 Model](#)
 - [4. Discussion](#)

1. Hardware and Software Specification

The hardware and software configuration of the testing platform is listed in Table 1.

Table 1: Hardware and Software Configuration

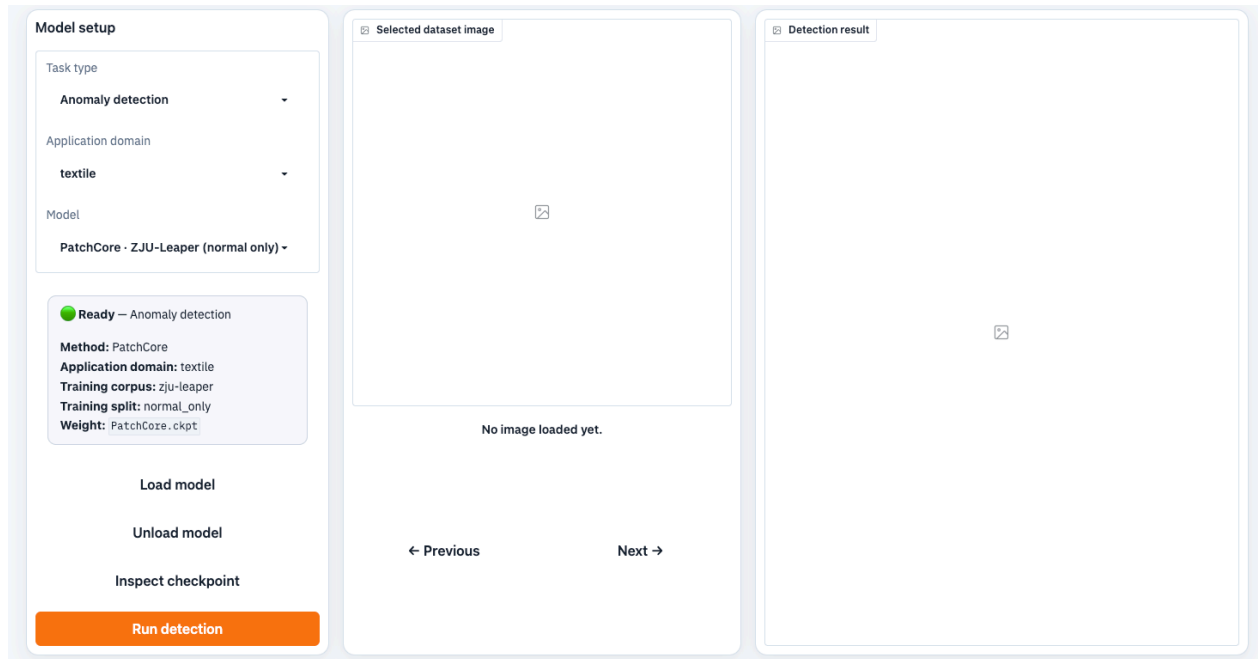
Subsystem	Value
Operating System	Ubuntu 22.04.5
CPU	2 × Intel Xeon Gold 6430
System Memory	256 GB
GPU & VRAM	NVIDIA A100 80GB
Python	3.12
PyTorch	2.14.0 (+cu130)
CUDA Toolchain	Driver 550.144.03, CUDA 12.4

2. Evaluation Reproduction

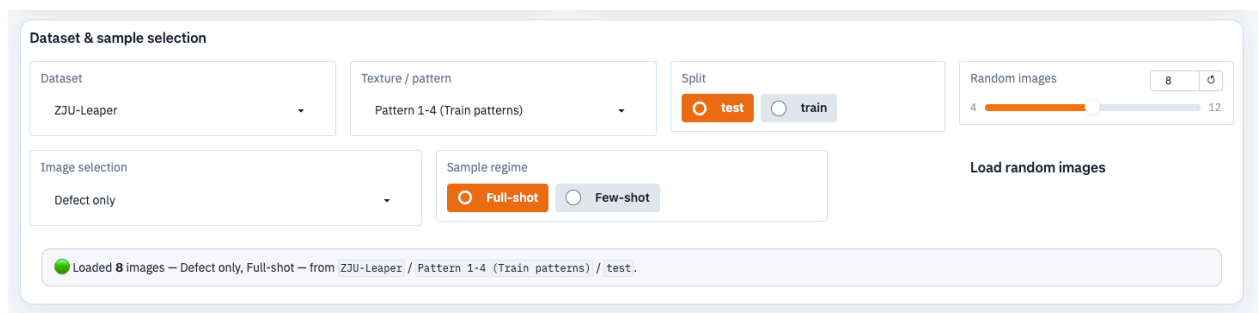
2.1 Examples

The environment was set up per [Environment Setup](#), then the test cases under [Examples](#) were reproduced.

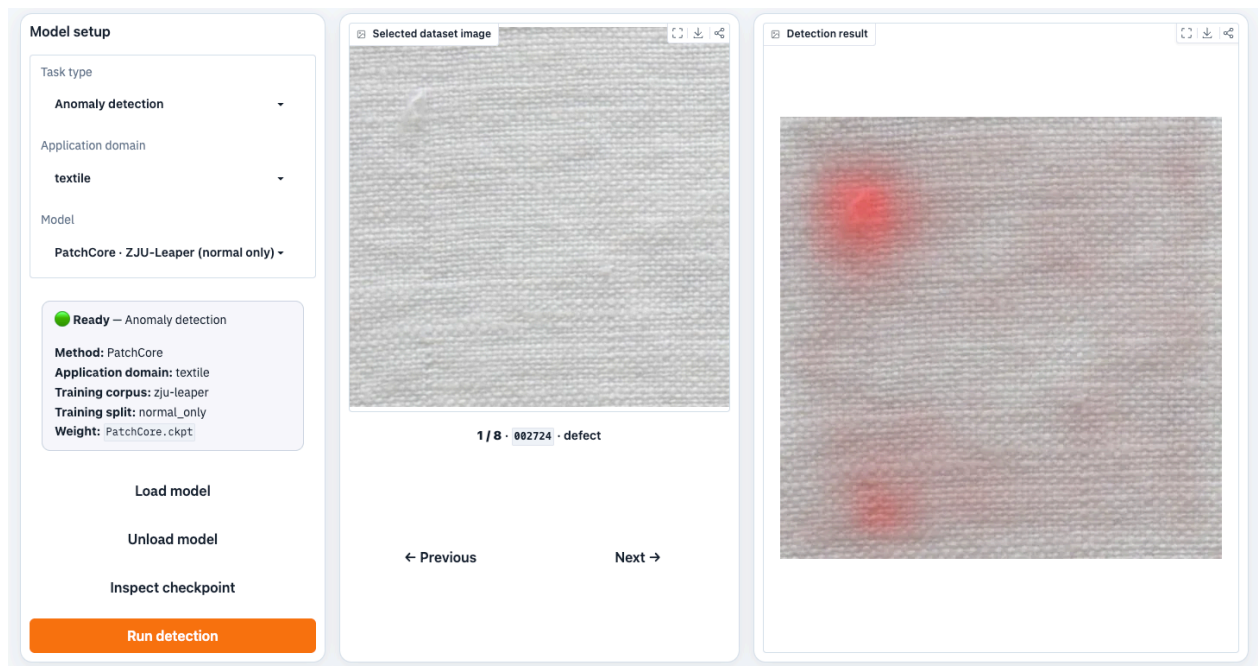
Following Steps 1 to 4, the target model was selected and loaded:



Following Step 5, the image to be evaluated was selected and loaded:



Following Step 6, anomaly detection was executed successfully. The resulting output confirms that the platform and its configuration are working correctly, so the remaining tests can be performed.



2.2 Technical and Overhead Metrics

Following the instructions in [README §4.1.3 Benchmarking](#), the complete benchmark suite was run. The resulting data are reported below as figures.

The evaluated configuration:

Item	Value
Dataset	ZJU-Leaper test split
Samples	350 in total
Input size	640 × 640
Precision	fp32
Metrics	Image, pixel, instance and overhead
Evaluated configurations	19

2.2.1 Technical Metrics: Image Level

This section evaluates image-level detection performance, i.e., whether an input image contains a defect. The metrics are defined in [README §3.3.1](#). Five representative metrics are selected for evaluation:

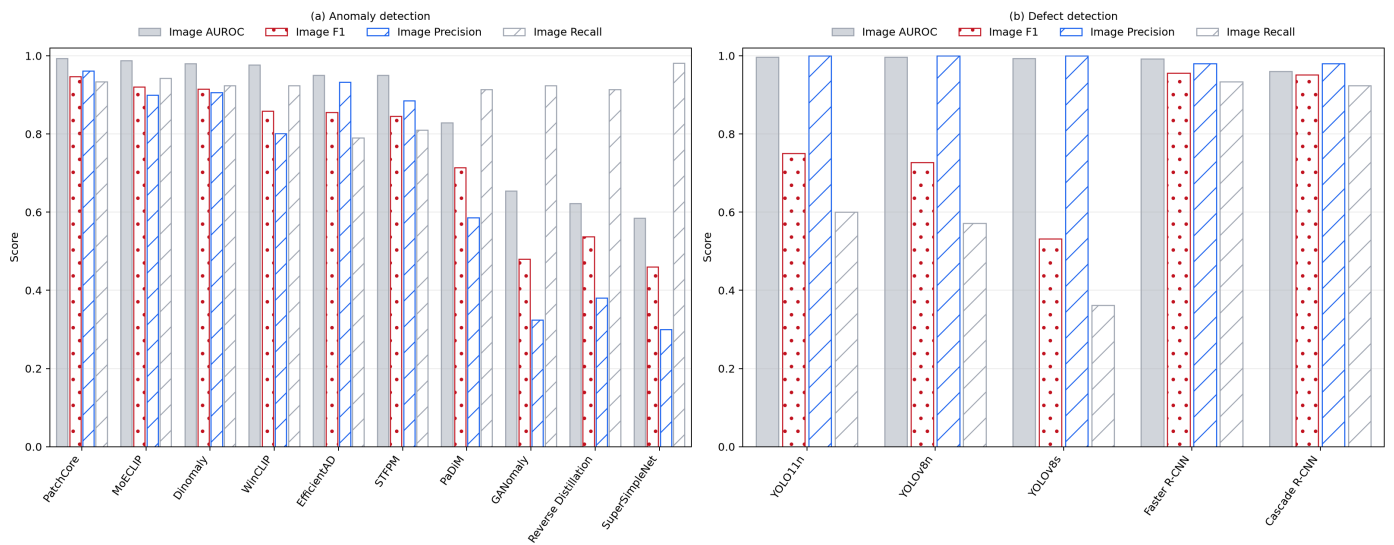
- **Image AUROC (Area Under the Receiver Operating Characteristic curve):** the probability that a defective image is ranked above a normal one. A value of 0.5 corresponds to random guessing, while 1.0 indicates perfect discrimination.
- **Image AP (Average Precision):** the area under the image-level precision-recall curve, summarising ranking performance across decision thresholds.
- **Image Precision:** the share of images flagged as defective that really are defective.
- **Image Recall:** the share of truly defective images that are flagged.

- **Image F1:** the balance of precision and recall in one number; it drops when either is poor.

Key observation:

- For the **defect detection models** shown on the right side of Figure 1, YOLO11n and YOLOv8n achieve the highest Image AUROC among the evaluated models, both reaching approximately 0.99. However, their recall is relatively lower at 0.600 and 0.571, respectively. This indicates that the two models produce few false-positive image-level predictions but miss a proportion of defective images.
- Among the **anomaly detection models**, PatchCore achieves the highest Image AUROC at 0.9932. In contrast, GANomaly, Reverse Distillation, and SuperSimpleNet obtain substantially lower Image AUROC values of 0.65, 0.62, and 0.59, respectively, indicating weaker image-level discrimination on this test set.

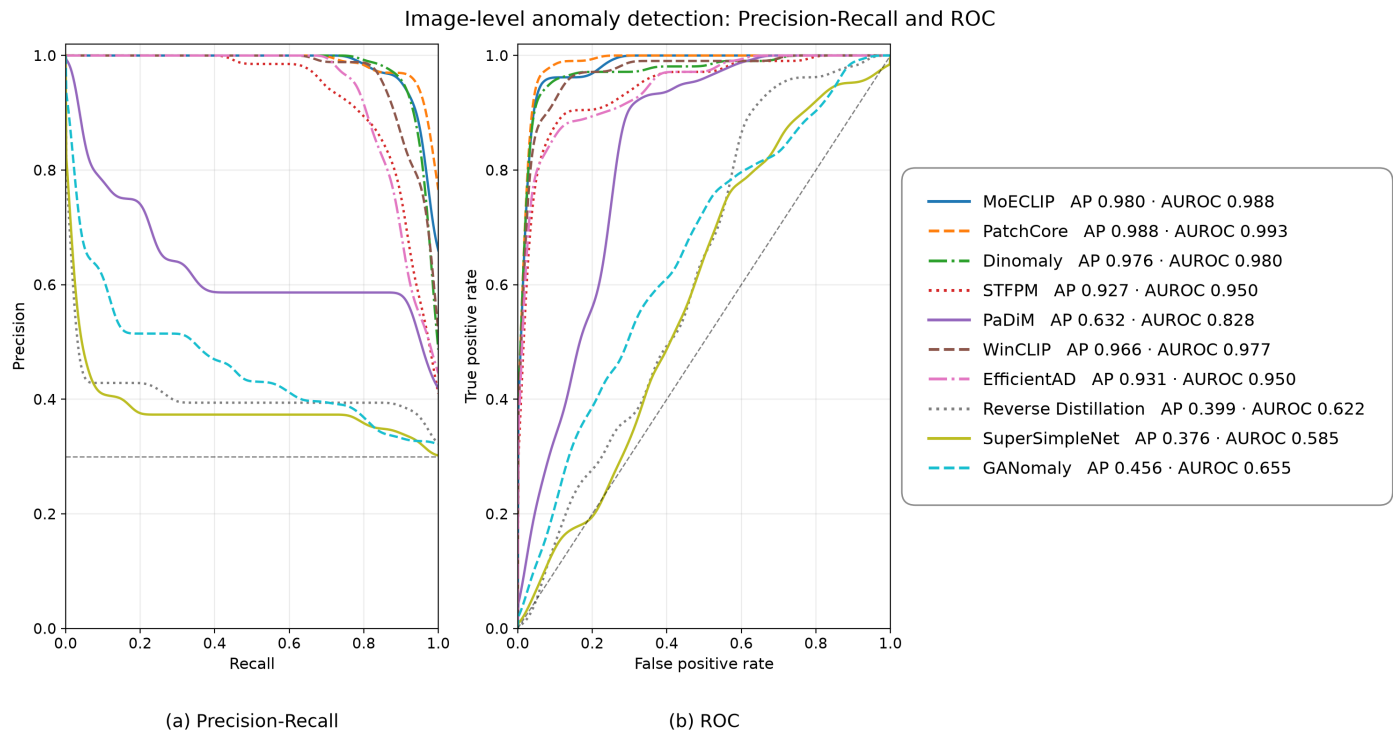
Figure 1: Image-level metrics



(a) Anomaly detection models; **(b)** defect detection models. Bars per model: (1) Image AUROC, (2) Image AP, (3) Image F1, (4) Image Precision and (5) Image Recall. An absent bar means that the corresponding model does not report that metric.

To further examine the image-level performance, Figure 2 shows the precision-recall (PR) and receiver operating characteristic (ROC) curves of the anomaly detection models. The PR curve illustrates the trade-off between precision and recall at different thresholds, while the ROC curve shows the trade-off between the true positive rate and false positive rate.

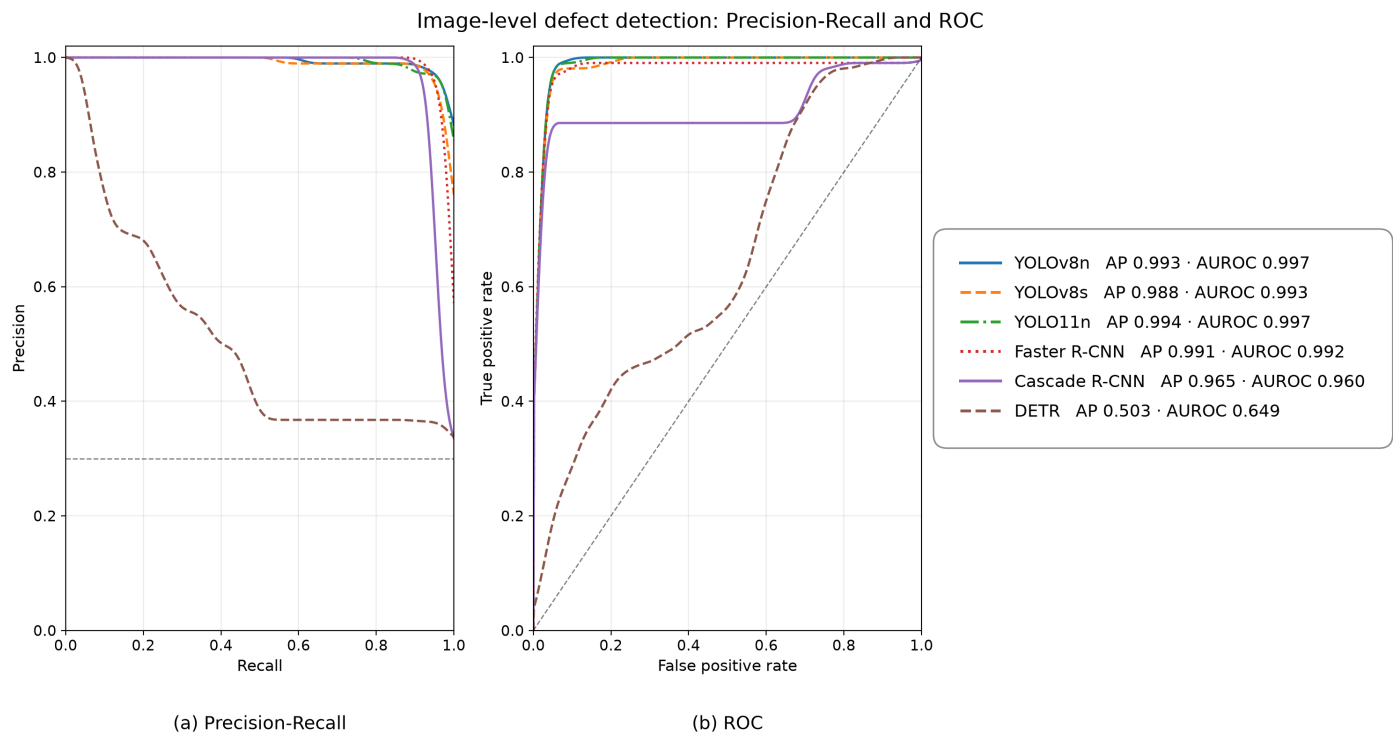
Figure 2: Image-level precision-recall and ROC, anomaly detection models



(a) Precision-recall curves; **(b)** ROC curves for the 10 anomaly detection models. Curves are smoothed for visualization only; AP and AUROC values are calculated from the original unsmoothed evaluation results.

Figure 3 shows the corresponding image-level PR and ROC curves for the defect detection models, providing a threshold-dependent view of their image-level classification performance.

Figure 3: Image-level precision-recall and ROC, defect detection models



(a) Precision-recall curves; (b) ROC curves for the 6 defect detection models. DETR is included because its image-level AUROC (0.65) and AP (0.50) can still be computed, although its instance-level detection results are degenerate.

2.2.2 Technical Metrics: Pixel Level

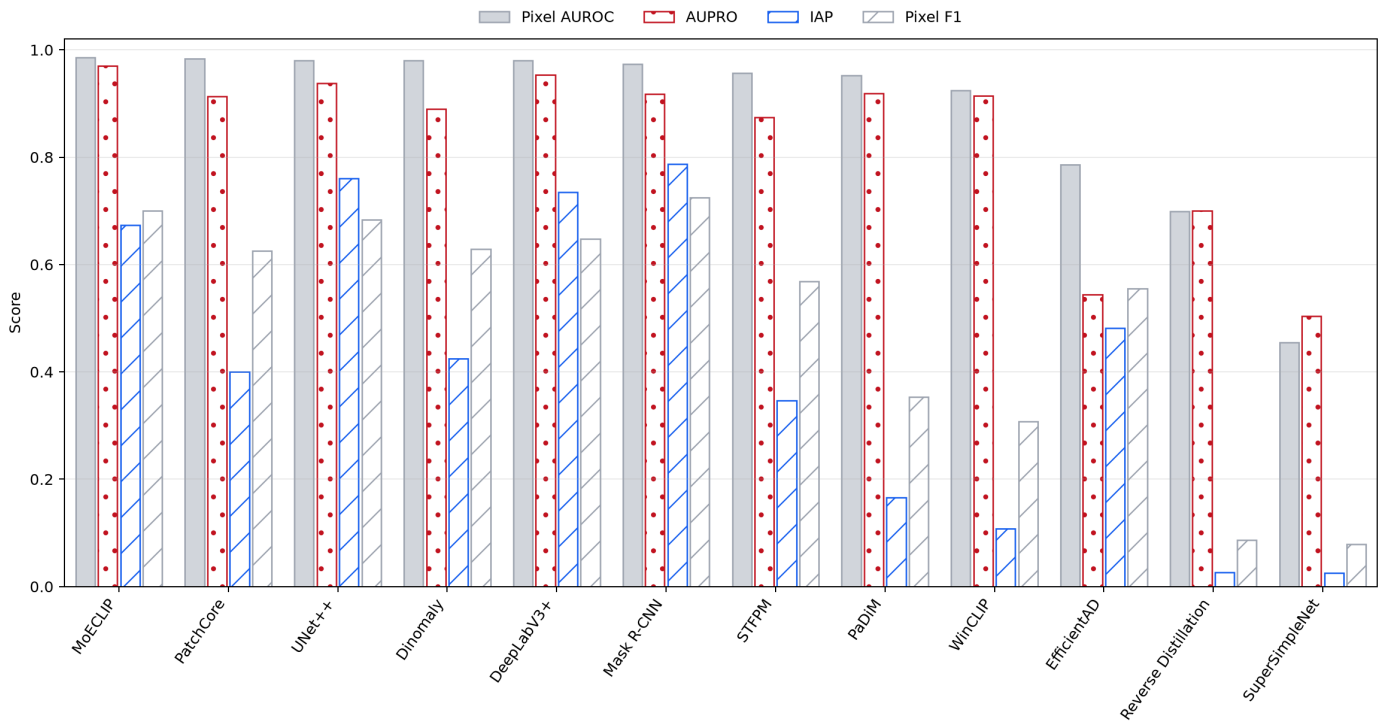
This section evaluates pixel-level localization performance, i.e., how accurately the defective pixels within an image are identified. The metrics are defined in [README §3.3.2](#). Four representative metrics are selected for evaluation:

- **Pixel AUROC:** the probability that a defective pixel is ranked above a normal one, pooled over the pixels of all images. A value of 0.5 corresponds to random guessing, while 1.0 indicates perfect discrimination.
- **Pixel AUPRO:** the share of each defective region that is covered before the false-positive rate reaches a fixed limit. Every region counts equally, so a small defect is not hidden by a large one.
- **Pixel F1:** the match between predicted and true defect pixels at a fixed pixel threshold, penalised by missed pixels and by extra pixels.
- **IAP (Instance Average Precision):** the average precision over connected defect regions, with every region weighted equally.

Key observation:

- Among the **anomaly detection models**, MoECLIP leads pixel-level localization, with 0.9857 Pixel AUROC and 0.9701 Pixel AUPRO, while SuperSimpleNet has the lowest values (0.4547 and 0.5033).
- Figure 4 also includes the three **segmentation models** because the current evaluation snapshot retains their pixel-level score maps and therefore reports the same four metrics for all 12 models. Among the segmentation models, Mask R-CNN has the highest Pixel F1, followed by UNet++ and DeepLabV3+.
- Pixel AUROC, AUPRO and IAP evaluate the ranking of continuous pixel scores over thresholds. Pixel F1 evaluates the binary prediction at the selected operating threshold and should therefore be interpreted together with the threshold-selection procedure.

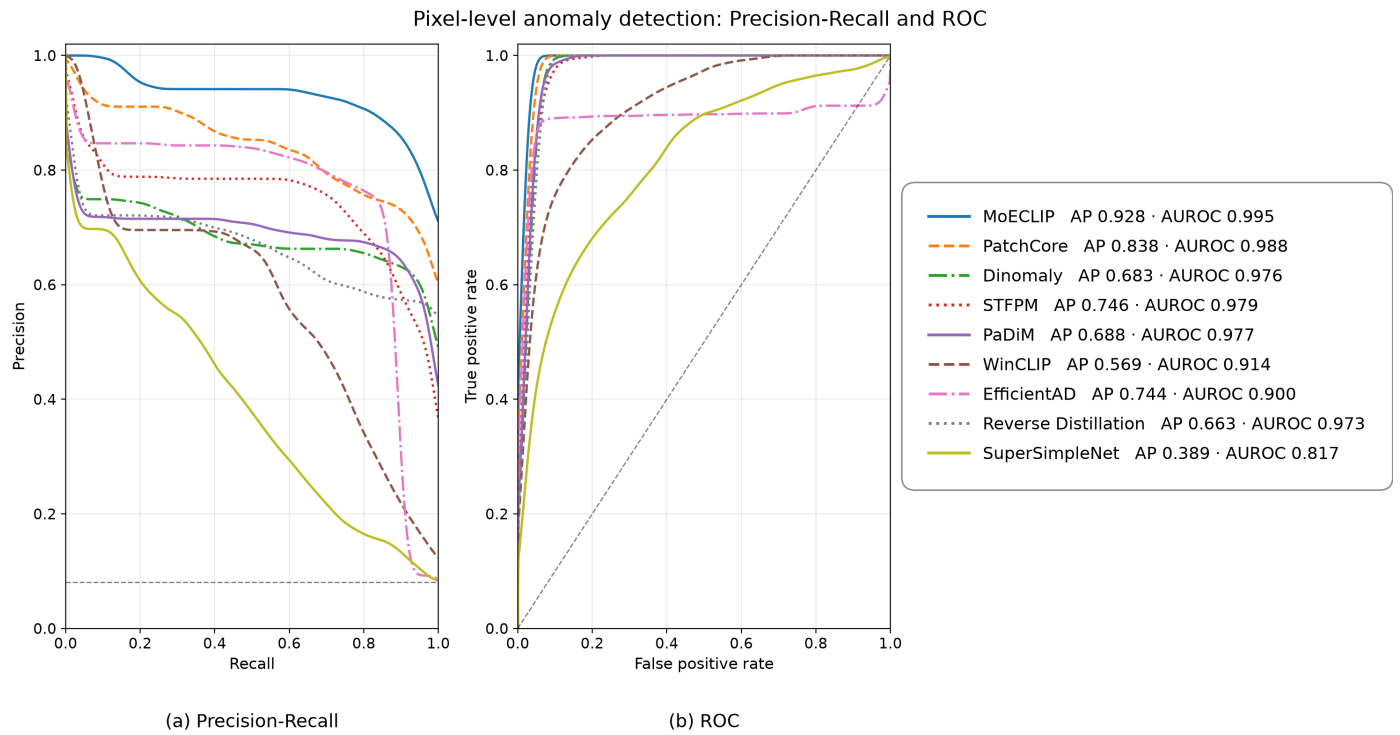
Figure 4: Pixel-level metrics on the A100 server



Bars per model: (1) Pixel AUROC, (2) AUPRO, (3) IAP and (4) Pixel F1. The figure includes nine anomaly detection models and three segmentation models. In the current snapshot, all 12 models retain a pixel-level score map and therefore report all four metrics. Pixel AUROC, AUPRO and IAP sweep a threshold over the score map; Pixel F1 evaluates the binary prediction at the selected operating threshold. mIoU is not plotted.

To further examine the pixel-level performance, Figure 5 shows the pixel-level precision-recall (PR) and receiver operating characteristic (ROC) curves of the anomaly detection models. The PR curve illustrates the trade-off between precision and recall at different thresholds, while the ROC curve shows the trade-off between the true positive rate and false positive rate.

Figure 5: Pixel-level precision-recall and ROC, anomaly detection models



(a) Precision-recall curves; (b) ROC curves for the 9 anomaly detection backends with persisted curve artifacts. The three segmentation models are summarised in Figure 4 but are outside the scope of this anomaly-model curve figure.

2.2.3 Technical Metrics: Instance Level

This section evaluates instance-level detection performance. The metrics are defined in [README §3.3.3](#). Six representative metrics are selected for evaluation:

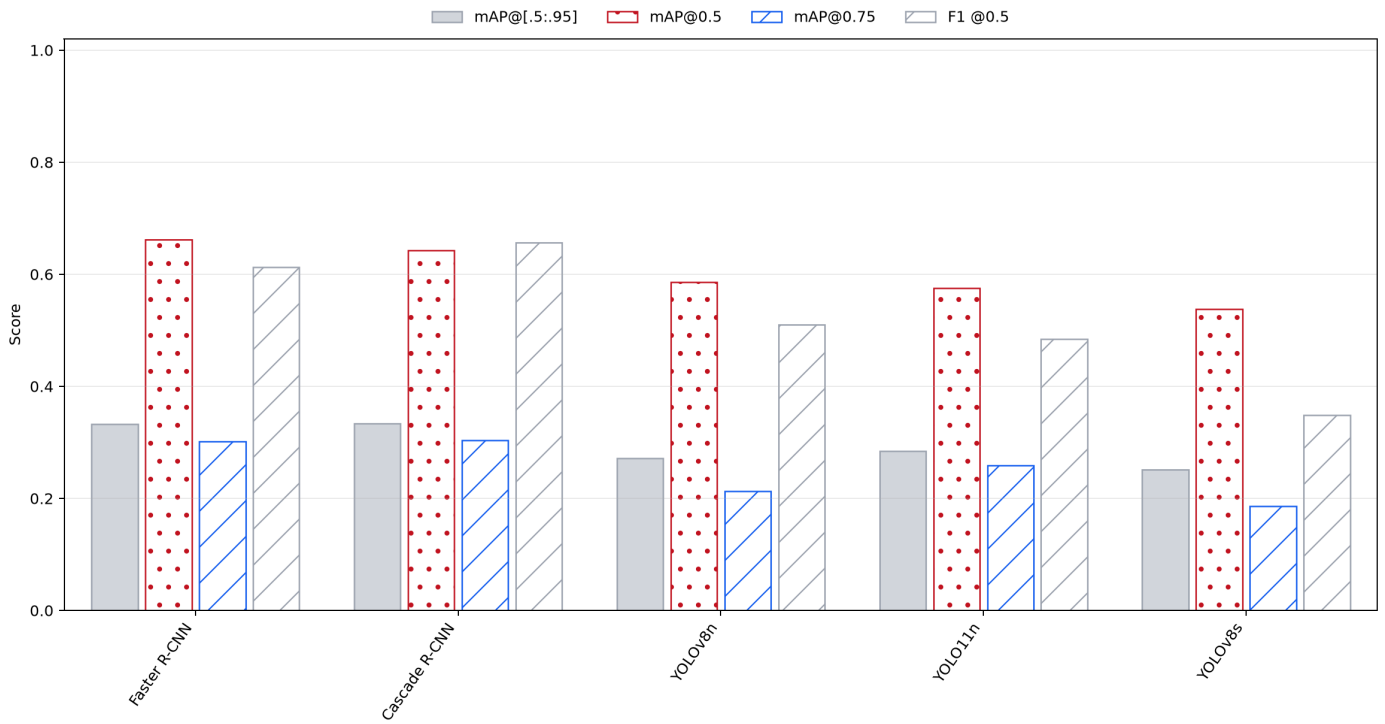
- **AP:** the box precision–recall quality averaged over IoU thresholds from 0.50 to 0.95 (COCO style); higher is better.
- **AP50 / AP75:** AP at a single IoU threshold of 0.50 or 0.75; 0.50 accepts a loosely placed box, while 0.75 demands a tight one, where IoU is the overlap between a predicted box and the ground-truth box.
- **Precision:** the share of reported boxes that overlap a real defect ($\text{IoU} \geq 0.50$).
- **Recall:** the share of real defects covered by a reported box.
- **F1:** the balance of box precision and recall in one number; it drops when either is poor.

Precision, Recall, F1 and the TP/FP/FN counts are reported at a confidence threshold of 0.25 and an IoU matching threshold of 0.50. Accordingly, Figure 6 labels this operating-point metric as F1@0.5; the 0.5 denotes the IoU threshold, not the confidence threshold.

Key observation:

- The **YOLO variants** form the precise-but-incomplete group: Precision reaches 0.857–0.929, but Recall stays at 0.214–0.363, so many defects are missed.
- The **two R-CNN detectors** show the opposite profile: higher Recall (0.692 and 0.681) at lower Precision (0.550 and 0.633). Cascade R-CNN achieves the best AP50 (0.6428) and F1 (0.6561).

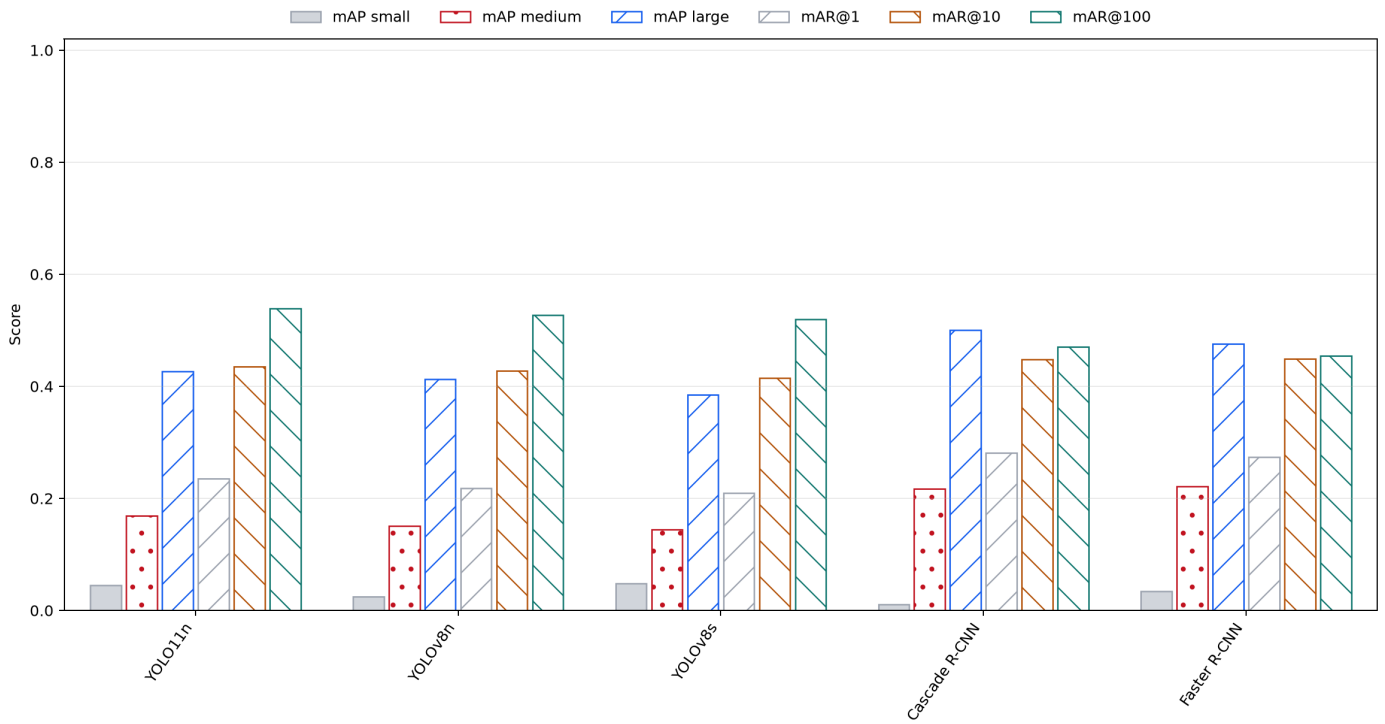
Figure 6: Instance-level detection metrics



Bars per model: (1) mAP@[.5:.95], (2) mAP@0.5, (3) mAP@0.75 and (4) F1@0.5. DETR is excluded from this bar chart because its mAP (0.0011) is below the figure's display floor and would not produce a readable comparison. Its value remains available in `snapshot_audit.csv`, and DETR is still included in the image-level results.

To further examine the instance-level performance, Figure 7 breaks AP down by the standard COCO small, medium and large object-area buckets. The mAR@k is the maximum recall when at most k detections are kept per image.

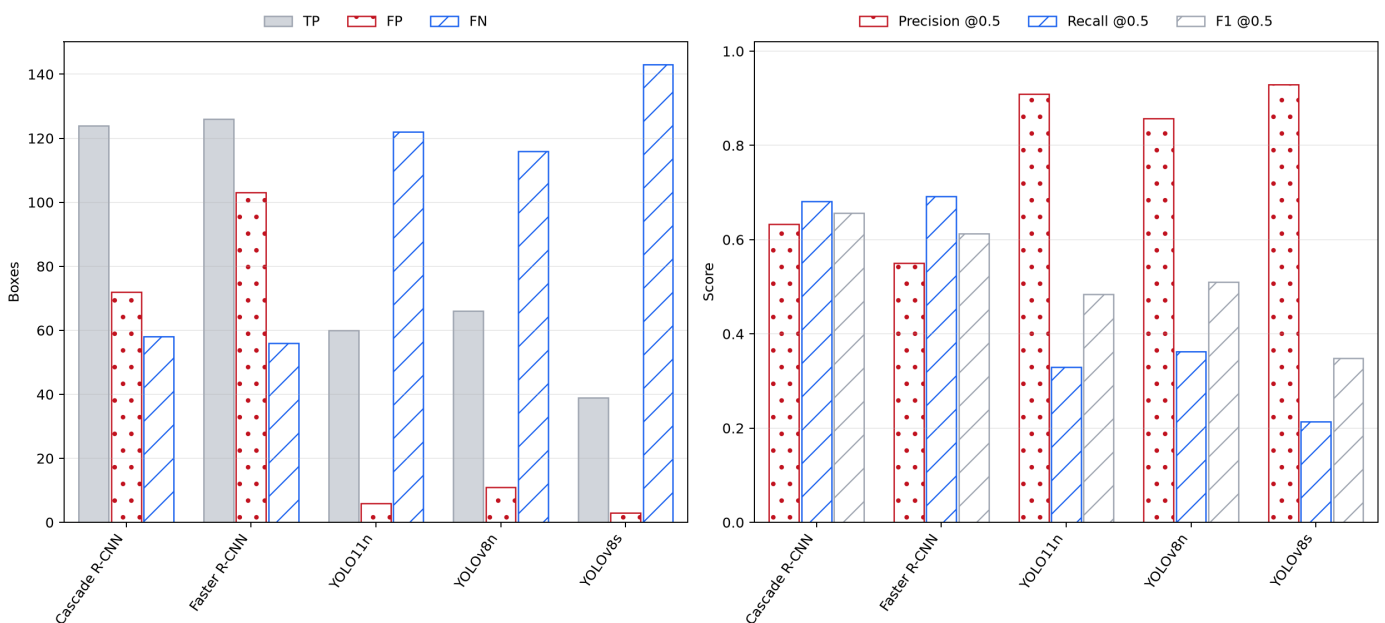
Figure 7: Size-bucketed AP and AR



Bars per model: (1) mAP for small defects, (2) mAP for medium defects, (3) mAP for large defects, (4) mAR@1, (5) mAR@10 and (6) mAR@100. Small, medium and large follow the standard COCO object-area buckets.

Figure 8 complements these rate-based metrics with a count-based view: the number of kept boxes shows whether a model finds the defects at all — something mAP alone does not say — while the Precision, Recall and F1 of those boxes quantify the corresponding trade-off.

Figure 8: Detections against ground truth



(a) Boxes kept at the evaluation confidence floor (0.25); (b) the Precision, Recall and F1 of those same boxes. DETR has no bars because it emits no box above the floor.

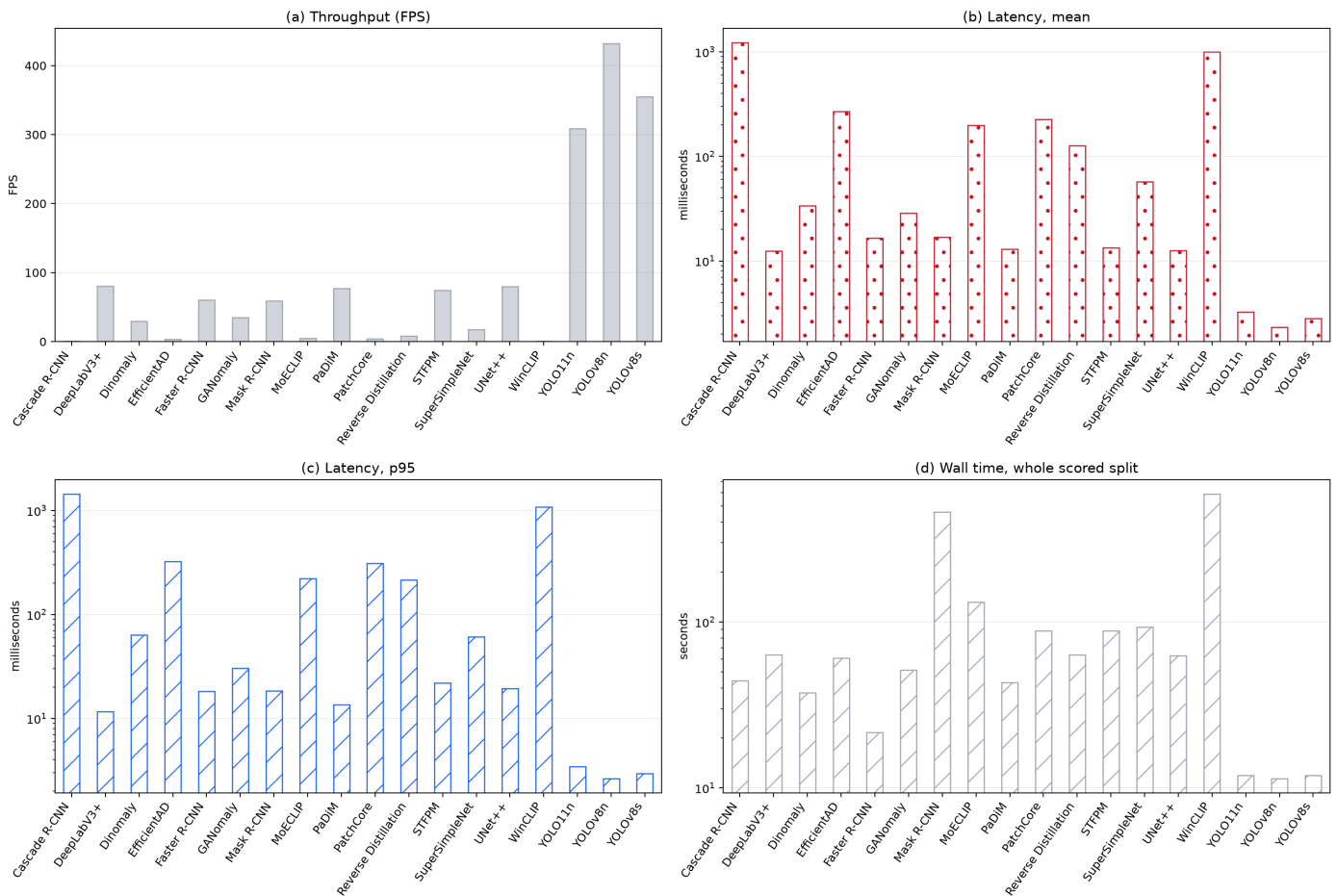
2.2.4 Overhead Metrics: Compute

This part evaluates the compute metrics defined by [README §3.3.5](#):

- **FPS:** images processed per second.
- **Latency mean / p95 / p99:** the time to process one image, in milliseconds; p95 and p99 describe tail latency rather than the average. Figure 9 shows the mean and p95; p99 remains available in the benchmark data but is not plotted.
- **FLOPs (G):** the arithmetic work of one forward pass, in billions of operations. Fixed by the architecture, so identical on any machine.
- **Wall-time:** the total seconds the scored run took, accuracy pass included.

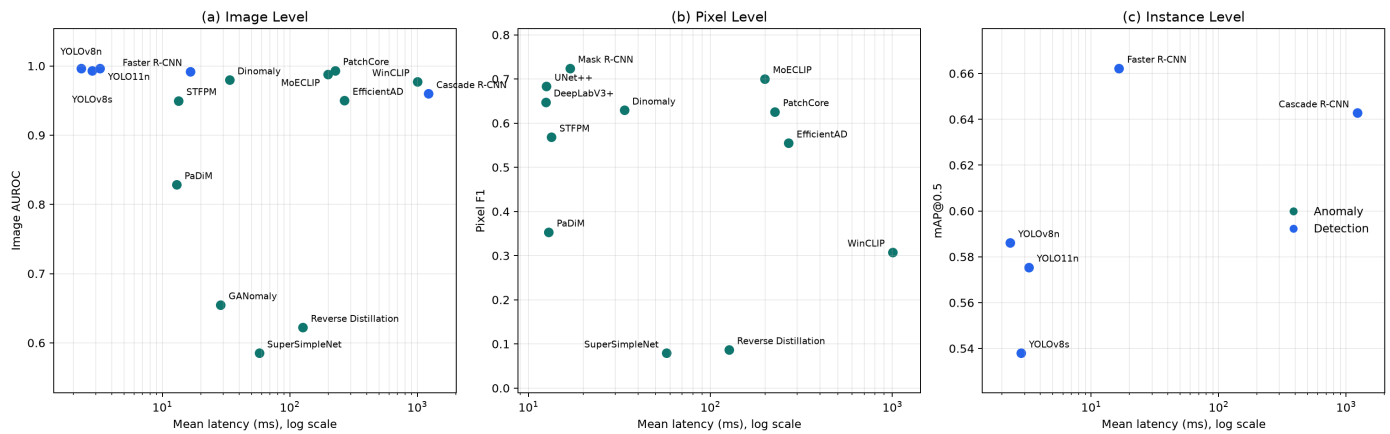
Key observation: Throughput spans more than three orders of magnitude on this machine: YOLOv8n reaches 432 FPS, while Cascade R-CNN reaches 0.82 FPS. FLOPs alone do not predict speed. YOLOv8s uses fewer FLOPs than YOLOv8n (6.99 G versus 14.90 G) but is slower (355 versus 432 FPS), showing that architecture and memory traffic also matter.

Figure 9: Compute cost on the A100 server



Four panels: (a) throughput (FPS), (b) mean latency, (c) p95 latency, (d) whole-split wall time. FPS is linear; latency and wall time are log-scaled because the backends span orders of magnitude. All 19 configurations are shown.

Figure 10: Quality versus latency, by metric granularity



Three panels by metric granularity rather than by model family: **(a)** image level (Image AUROC), **(b)** pixel level (Pixel F1), **(c)** instance level (mAP@0.5); mean latency is on a log axis. Anomaly detection and segmentation models share panel (b), which is the only quality metric they both report.

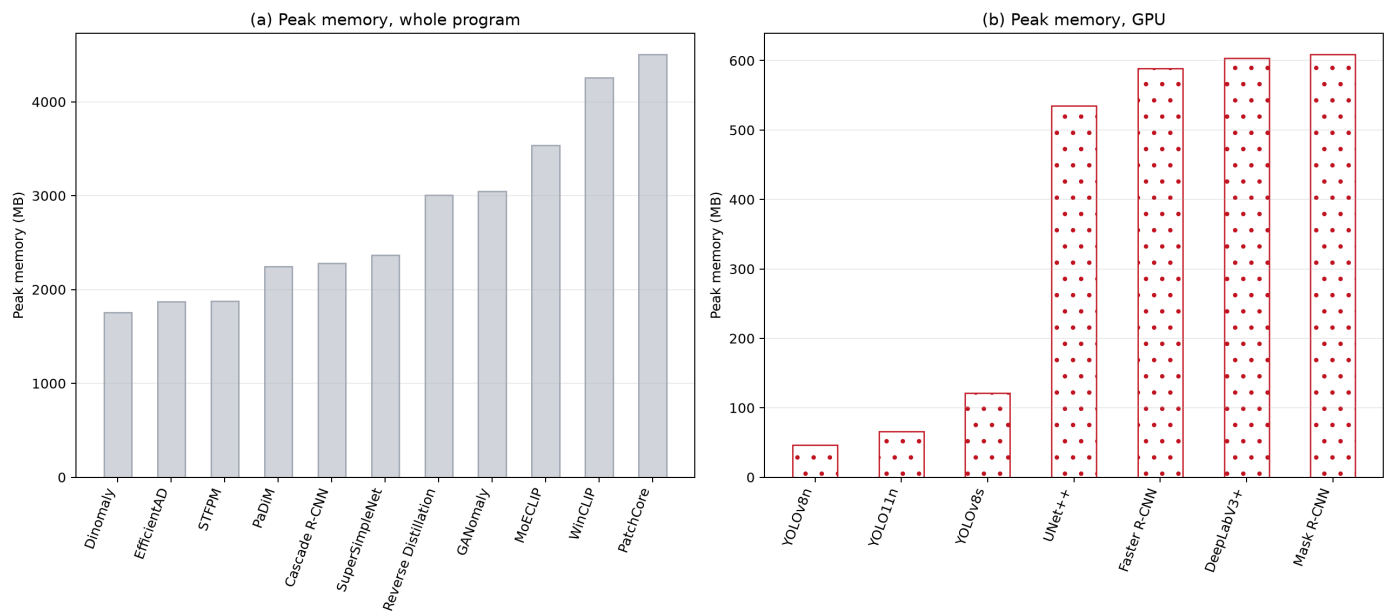
2.2.5 Overhead Metrics: Memory

This part evaluates the memory metrics defined by [README §3.3.6](#):

- **Parameters (M):** the number of learned weights, in millions. Fixed by the architecture, so identical on any machine.
- **Peak memory:** the most memory used at once during the run. **Two different instruments produced this number, and which one applies to a model depends on how that model could be profiled** — see the note under Figure 11.
- **Allocator peak:** peak tensor memory live on the graphics card, which excludes the Python interpreter, host copies and the CUDA context itself.
- **Artifact size:** the checkpoint file on disk (weights plus, for a fine-tuned model, its optimizer state). This is storage, not run-time memory.

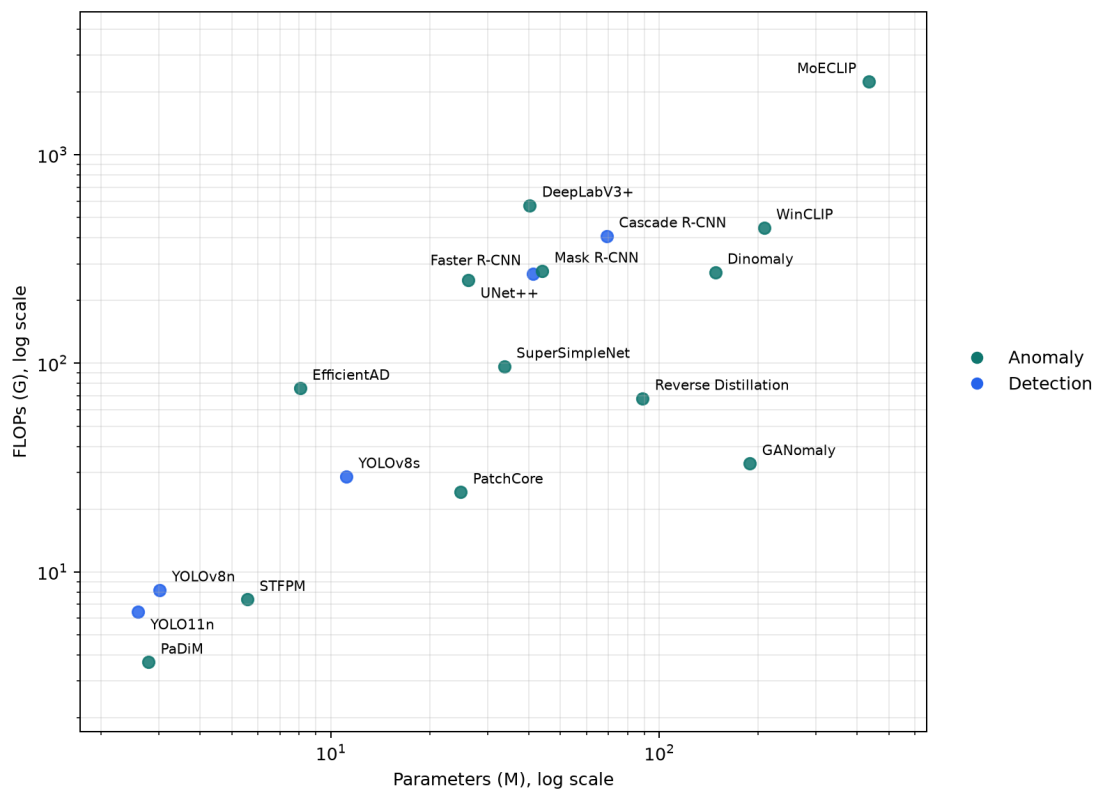
Key observation: The two peak-memory measurements are not interchangeable and do not form a single model ranking. In the GPU-allocator panel, YOLOv8n has a measured tensor-memory peak of 46.5 MB. In the whole-process panel, PatchCore has the highest measured resident-memory peak at 4505 MB. These values must be interpreted only within their respective panels: whole-process RSS includes weights, the CUDA context, host copies and framework overhead, whereas the CUDA allocator counts only live GPU tensor memory. Figure 11 therefore separates the two instruments and gives each panel its own axis.

Figure 11: Peak memory by measurement instrument



(a) Peak memory, whole program and **(b) Peak memory, GPU**. The two panels are **not two views of one number** and must not be compared across: panel (a) counts the entire Python process (weights, graphics-card context, host copies, framework overhead) while panel (b) counts tensor memory live on the card — a model can show 1.8 GB in (a) and 0.05 GB in (b) because its weights live in host memory. Each panel has its own y-axis, and a model appears only in the panel whose instrument measured it: 11 models in (a), 8 in (b). Which instrument a model got is decided by whether it could be exported for profiling, not by its accuracy or its size; the main report's memory section explains this and lists the instrument per model. Peak memory is a property of this host and should not be carried to another machine.

Figure 12: Model size and compute complexity

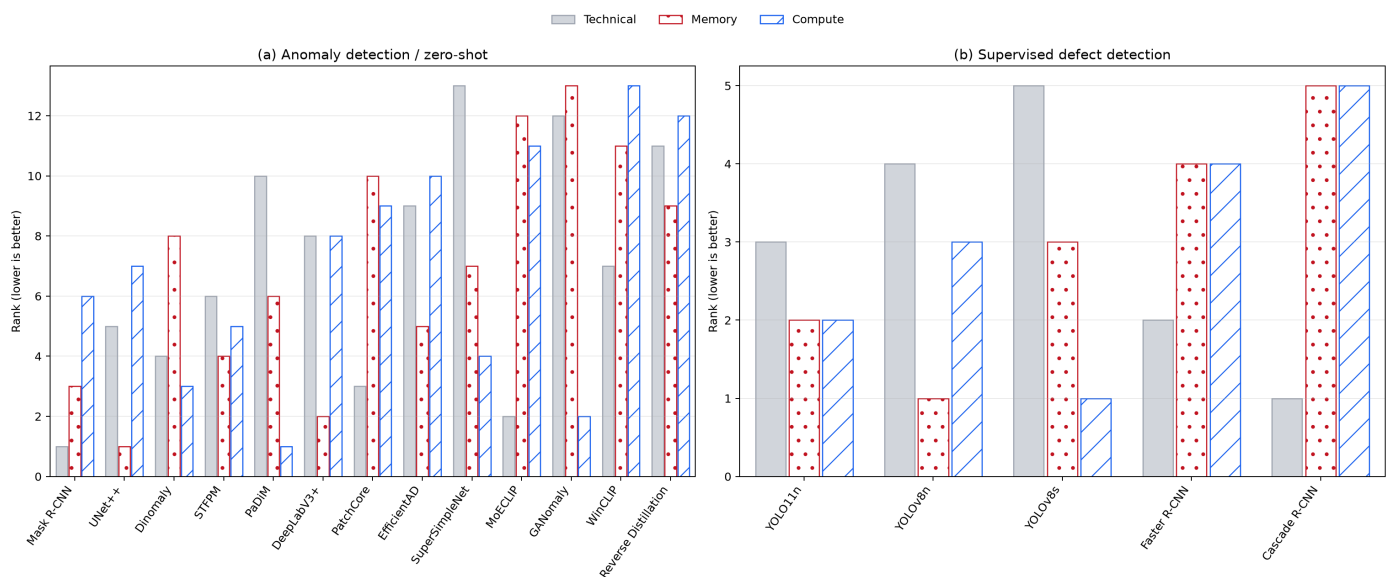


Parameters against FLOPs, both log-scaled and annotated. Both are fixed by the architecture, so unlike FPS, latency and memory they transfer to any machine. Colour marks the family: anomaly (teal) or detection (blue).

2.3 Analysis

The 19 configurations cover three model groups: **anomaly detection (AD)**, **defect detection (DD)**, and **segmentation (SEG)**. They are compared on the metric families in [README §3.3](#): image level, pixel level, instance level, and overhead. Figure 13 reports separate technical, memory, and compute ranks; models are compared only with models that report the same metrics.

Figure 13: Three-dimensional ranking



(a) anomaly detection / zero-shot, **(b)** supervised defect detection — split because an anomaly model and a detector were never measured on the same metrics. Each model gets three bars: its rank within its own panel on **Technical** (the image + pixel + instance accuracy tables), **Memory** and **Compute** (the two overhead tables). Every metric is scored in the direction the project's metric taxonomy declares for it (`higher` / `lower`; `neutral` and non-numeric columns are skipped), min-max normalized across the panel, and a model's dimension score is the average of the metrics it actually reports — so reporting fewer metrics is neither punished nor rewarded. All three bars are ranks (1 = best) on one axis; models are ordered by their mean rank, which is not drawn, because a fourth series on a rank axis reads as a fourth measurement rather than as a summary. Panel (a) includes the three segmentation models, which the project classifies as pixel-level anomaly detection, so it ranks 13 models rather than the 10 anomaly backends alone.

- **PatchCore**: strongest anomaly image score (AUROC 0.9932) and good localization; its weakness is very high memory use.
- **MoECLIP**: best pixel localization (AUROC 0.9857, AUPRO 0.9701) and strong image-level performance; it is memory-intensive and not the fastest option.
- **Dinomaly**: balanced image- and pixel-level results; it does not lead either metric family.

- **EfficientAD**: moderate image-level performance with relatively low overhead; its pixel localization trails the leading anomaly detection models.
- **PaDiM**: relatively low compute cost; its image and pixel scores are weaker than the leading anomaly detection models.
- **STFPM**: balanced image-level precision and recall with solid pixel AUROC; its pixel F1 remains modest.
- **WinCLIP**: strong image-level ranking with modest accuracy elsewhere; its throughput is close to 1 FPS.
- **GANomaly, Reverse Distillation, and SuperSimpleNet**: lower image-level performance; SuperSimpleNet also has the weakest pixel localization.
- **YOLO11n and YOLOv8n**: highest image AUROC (0.9965), high precision, and real-time throughput; their recall is limited, so they miss some defects.
- **YOLOv8s**: maintains high precision but is slower and has lower recall than the smaller YOLO variants.
- **Faster R-CNN and Cascade R-CNN**: higher recall and better instance completeness than YOLO; they require substantially more compute, and Cascade R-CNN is the slowest detector.
- **DETR**: produces no valid detection at the evaluation threshold and is therefore unsuitable for this test configuration.
- **Mask R-CNN, UNet++, and DeepLabV3+**: provide binary segmentation masks. Mask R-CNN has the best Pixel F1 (0.7242), followed by UNet++ (0.6833) and DeepLabV3+ (0.6477); their output does not support threshold-free pixel curves.

Overall, the choice depends on the deployment goal: YOLO11n or YOLOv8n for fast, precise screening; R-CNN models when recall is more important; MoECLIP for pixel localization; and PatchCore when image-level anomaly ranking is the priority.

Scope. Every number here describes this host. Parameter counts and FLOPs are fixed by the architecture and transfer to any machine; FPS, latency and peak memory do not.

3. Extensibility

3.1 Adding a New Dataset

Following [README §6.1](#), the `bottle` category of MVTec AD was copied into a new dataset root, `datasets/general/Bottle`, and PatchCore was trained and tested on it.

Table 15: Extensibility validation — PatchCore on the newly added Bottle dataset

Image AUROC	Pixel AUROC	Pixel AUPRO	IAP
1.0000	0.9935	0.9934	0.8461

The results are usable, confirming that the new dataset was added successfully and that PatchCore trained on it.

3.2 Adding the YOLO 26 Model

Following [README §6.2](#), YOLO 26 was added to the `ultralytics` backend:

- **Preset entry** — `yolo26n` variant and alias in `presets.py`
- **Model configuration** — `configs/models/ultralytics_yolo26_example.yaml`
- **Registry row** — one `yolo26n` entry in `configs/registry/models.yaml`
- **Training run** — ZJU-Leaper, test mode (a pipeline check, not an accuracy run): 1 epoch, batch 2, 640×640 input, 8

training and 8 validation samples

The run produced a new model, registered as `textile/artifacts/models/yolo26n_yolo26n_zju_leaper.pt` with 2.50 M parameters, confirming that YOLO 26 was added successfully.

4. Discussion

- **Training budgets are short.** Several entries were trained for only a few steps — the anomaly smoke configuration is 1 epoch on 8 images — so the weaker results in §2.3 reflect the run budget as much as the architecture.
- **Overhead metrics are host-defined.** FPS, latency and peak memory are properties of the machine that measured them — this run is an A100 server — so they should not be carried to another host. Only FLOPs and parameter counts are host-independent.
- **Coverage gaps.** MambaAD untested; MoECLIP scored on MVTec AD rather than ZJU-Leaper; cross-domain ([README §3.3.4](#)) not measured on this run; `LMEI`, `Max streams @budget`, `1-stream latency`, `resolution slope` and power/energy not produced.